

TUGAS MATA KULIAH TEKNOLOGI IOT
LAPORAN PROGRES PROJECT TERPADU
SMART COFFEE PROCESSING: IOT-BASED ELECTRONIC NOSE
AND TINYML



Dosen Pengampu: Ahmad Radhy, S.Si., M.Si.

Disusun Oleh:

Grup A

Natasya Putri Regina (2042241026)

Ghani Raihan Syakir (2042241036)

Grup C

Evan Javier Firdausi Malik (2042241010)

Mahendra Dwi Fahreza (2042241023)

PROGRAM STUDI D4 TEKNIK REKAYASA INSTRUMENTASI
DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

Ringkasan Eksekutif

Laporan terpadu ini menyajikan perancangan dan implementasi sistem instrumentasi cerdas *end-to-end* dalam proyek “**Smart Coffee Processing: IoT-Based Electronic Nose and TinyML**”. Sistem dirancang untuk mengklasifikasikan profil aroma biji kopi Arabika dan Robusta secara objektif di tingkat perangkat tepi (*edge computing*) menggunakan mikrokontroler ESP32-S3 yang diprogram dengan bahasa **Embedded Rust** (`#![no_std]`) berbasis kerangka asinkron Embassy.[1, 2]

Model kecerdasan buatan dilatih via Edge Impulse dan dikuantisasi ke format integer 8-bit (*int8*) untuk inferensi mandiri di perangkat (*On-Device TinyML*). Telemetri data dikirimkan melalui protokol MQTT 3.1.1 menuju *cloud backend* berbasis Rust Axum dan database PostgreSQL, divisualisasikan pada dashboard *real-time*, serta didukung oleh mekanisme manajemen perangkat jarak jauh termasuk pembaruan perangkat lunak jarak jauh (*OTA Firmware Update*) via platform ThingsBoard.[5, 4]

Contents

Ringkasan Eksekutif	i
1 Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Pengembangan	1
2 Identitas Sistem dan Spesifikasi Proyek	2
3 Arsitektur Sistem Terintegrasi	3
3.1 Gambaran Umum Sistem	3
3.2 Alokasi Pin Perangkat Keras (<i>Hardware Pinout Mapping</i>)	3
4 Pengembangan Model TinyML pada Edge Impulse	5
4.1 Akuisisi Data Live dan Koneksi Perangkat (<i>Live Data Ingestion</i>)	5
4.2 Konfigurasi Impulse Design dan Grafik Kanal Sensor	5
4.3 Arsitektur Model Neural Network dan Deployment On-Device	6
4.4 Implementasi Inferensi On-Device di Embedded Rust	7
5 Integrasi Jaringan dan Telemetry Cloud MQTT	8
5.1 Protokol Komunikasi Jaringan	8
5.2 Validasi Payload dan Skema Backend	8
6 Implementasi dan Progres Terbaru Backend & Cloud	9
6.1 Implementasi Backend dan REST API	9
6.2 Database, Dataset, dan Seeder	9
6.3 Dashboard, PDF Reporting, Docker & Deployment	9
7 Dokumentasi Visual, Cloud Deployment & OTA ThingsBoard	10
7.1 Visualisasi Real-Time Dashboard Web	10
7.2 Analytical Report PDF Otomatis	10
7.3 Arsitektur Layanan dan Variabel Lingkungan di Railway Cloud	11

7.4	Setup OTA (Over-The-Air) Update pada ThingsBoard dan Status Sukses	13
8	Tantangan yang Dihadapi dan Solusi (<i>Engineering Challenges</i>)	15
9	Kesimpulan	16
A	Lampiran Teknis Proyek	17
A.1	Persiapan Environment	17
A.2	Clone dan Setup Project	17
A.3	Konfigurasi .env	17
A.4	Docker dan Deployment	17
A.5	Struktur Repository	17

BAB 1

Pendahuluan

1.1 Latar Belakang

Internet of Things memungkinkan perangkat fisik mengakuisisi data, mengirimkannya melalui jaringan, dan menghubungkan data tersebut dengan sistem pengolahan serta penyimpanan. Pada project E-Nose, data diperoleh dari beberapa sensor gas dan diarahkan untuk analisis pola aroma kopi. README root menyebut ESP32-S3 sebagai *edge device*, Edge Impulse sebagai framework ML, MQTT sebagai protokol komunikasi, dan backend cloud sebagai pusat ingestion serta monitoring.[1]

Selain telemetry sensor, pengelolaan siklus hidup perangkat keras di lapangan memerlukan mekanisme pemeliharaan jarak jauh yang andal. Integrasi platform manajemen IoT seperti ThingsBoard memungkinkan pengelola sistem melakukan pembaruan firmware jarak jauh (*Over-The-Air OTA Update*) secara aman dan terstruktur tanpa intervensi fisik langsung pada perangkat E-Nose.[3]

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan Embedded Rust (`#![no_std]`) pada ESP32-S3 untuk akuisisi sensor dan inferensi TinyML secara mandiri?
2. Bagaimana melakukan akuisisi data live multi-kanal sensor dan melatih model klasifikasi pada platform Edge Impulse?
3. Bagaimana menerima data measurement dari E-Nose melalui protokol REST API dan MQTT?
4. Bagaimana menyimpan measurement secara terstruktur pada SQLite maupun PostgreSQL di Cloud?
5. Bagaimana mengonfigurasi manajemen perangkat dan melakukan pembaruan firmware jarak jauh (OTA Update) menggunakan ThingsBoard?

1.3 Tujuan Pengembangan

Membangun fondasi sistem IoT terpadu berbasis perangkat keras ESP32-S3 dan cloud backend yang mampu melakukan inferensi TinyML secara mandiri, menerima telemetry MQTT/REST, menyajikan dashboard, menghasilkan laporan PDF, serta mendukung pembaruan sistem secara jarak jauh (*OTA Firmware Update*).[5, 3]

BAB 2

Identitas Sistem dan Spesifikasi Proyek

Spesifikasi teknis sistem instrumentasi yang dibangun dirangkum dalam Tabel 2.1.

Table 2.1: Spesifikasi Teknis Sistem Instrumentasi E-Nose dan TinyML

Komponen / Parameter	Spesifikasi Implementasi
Nama Proyek	Smart Coffee Processing: IoT-Based Electronic Nose and TinyML
Target Perangkat Keras	Espressif ESP32-S3 (Xtensa Dual-Core 32-bit LX7, 240 MHz, 8 MB Flash, 512 KB SRAM)
Bahasa & Lingkungan	Rust 2021 Edition, Bare-Metal (<code>#![no_std]</code>), <code>esp-hal</code> v0.22.0, <code>esp-wifi</code> v0.11.0
Asynchronous Runtime	Embassy Executor (<code>esp-hal-embassy</code> v0.5.0, <code>embassy-net</code> v0.4.0)
Larik Sensor Gas (MOS)	8 Kanal: MQ-6, MQ-135, MQ-3, MQ-7, TGS-2611, TGS-2602, TGS-2600, TGS-2620
Antarmuka ADC	2x ADS1115 (16-bit Precision ADC, Alamat I2C: 0x48 dan 0x49)
Platform Manajemen IoT	ThingsBoard IoT Platform (Device Provisioning, Telemetry, & OTA Updates)
Versi Firmware OTA	<code>enose-firmware</code> (v2.0.0)
Platform TinyML	Edge Impulse Studio (<code>ghani-afk-project-1</code>), Model Neural Network Keras
Protokol Komunikasi	Wi-Fi Station (802.11 b/g/n, WPA2-PSK), MQTT 3.1.1 (TCP Port 1883)
Cloud Backend	Rust Axum Web Service (<code>enose-cloud-backend</code>) di-deploy pada Railway.app

BAB 3

Arsitektur Sistem Terintegrasi

3.1 Gambaran Umum Sistem

Sistem mengadopsi model berlapis (*layered architecture*) yang menghubungkan fenomena fisika-kimia aroma kopi hingga visualisasi analitik pada dashboard cloud dan manajemen OTA ThingsBoard, sebagaimana diuraikan pada Gambar 3.1.

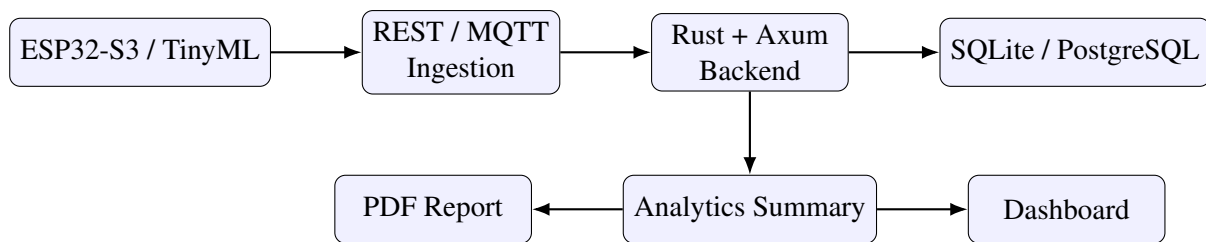


Figure 3.1: Arsitektur sistem berbasis implementasi backend dan konteks arsitektur project

3.2 Alokasi Pin Perangkat Keras (*Hardware Pinout Mapping*)

Implementasi firmware mengatur 14 pin I/O fisik ESP32-S3 secara terisolasi tanpa menimbulkan konflik bus serial, sebagaimana disajikan pada Tabel 3.1.

Table 3.1: Pemetaan Pin GPIO Mikrokontroler ESP32-S3

Pin GPIO	Mode Konfigurasi	Fungsi Alokasi Sistem
GPIO 8	Open-Drain / I2C0	I2C SDA (Data Serial untuk 2x ADC ADS1115)
GPIO 9	Output / I2C0	I2C SCL (Clock Serial untuk 2x ADC ADS1115)
GPIO 10	Output Open-Drain	Saluran Data Sensor Suhu/Kelembapan Lingkungan DHT22 #1
GPIO 11	Output Open-Drain	Saluran Data Sensor Suhu/Kelembapan Lingkungan DHT22 #2
GPIO 5	Digital Output	Driver Motor A – Input IN1 (Pompa Inlet Ali-ran Sampel)
GPIO 17	Digital Output	Driver Motor A – Input IN2 (Katup Solenoid Inlet)
GPIO 14	Digital Output	Driver Motor A – Input IN3 (Auxiliary Control Inlet)

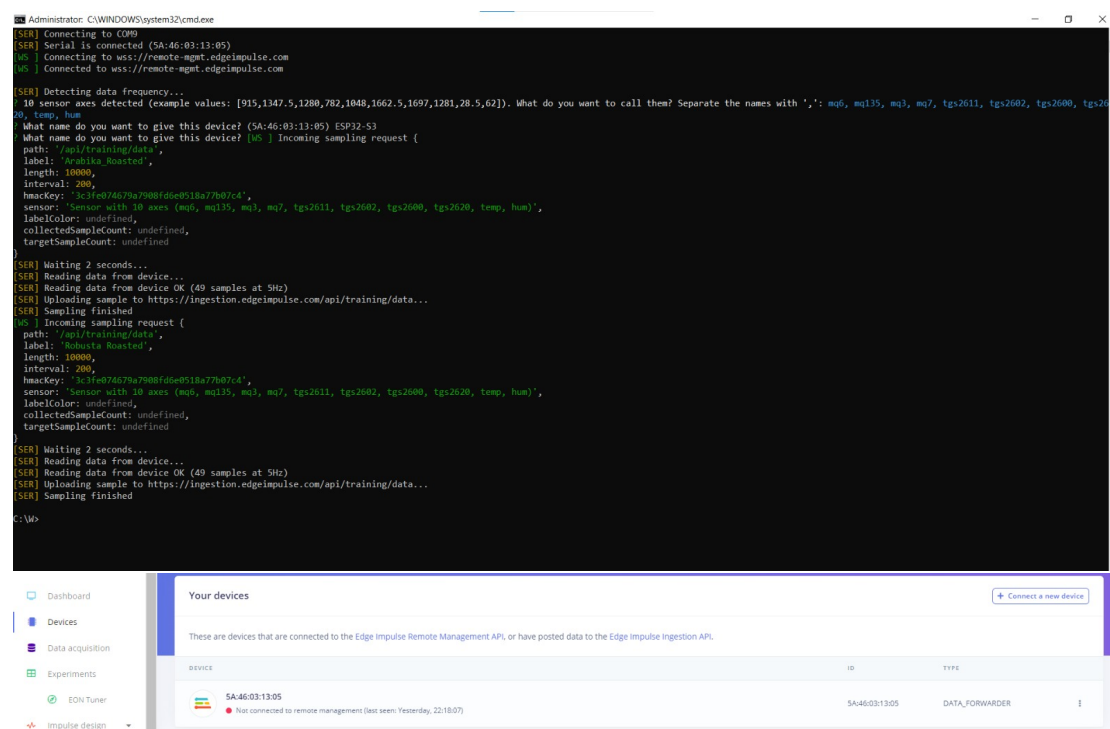
Pin GPIO	Mode Konfigurasi	Fungsi Alokasi Sistem
GPIO 19	Digital Output	Driver Motor A – Input IN4 (Auxiliary Control Inlet)
GPIO 16	Digital Output	Driver Motor B – Input IN1 (Pompa Outlet Ali-ran Pembersih)
GPIO 4	Digital Output	Driver Motor B – Input IN2 (Katup Solenoid Outlet)
GPIO 45	Digital Output	Driver Motor B – Input IN3 (Auxiliary Control Outlet)
GPIO 20	Digital Output	Driver Motor B – Input IN4 (Auxiliary Control Outlet)
GPIO 43	Peripheral UART0	UART0 TX (Komunikasi Serial Data / Edge Impulse CLI)
GPIO 44	Peripheral UART0	UART0 RX (Komunikasi Serial Perintah / CLI)

BAB 4

Pengembangan Model TinyML pada Edge Impulse

4.1 Akuisisi Data Live dan Koneksi Perangkat (*Live Data Ingestion*)

Akuisisi data dilakukan dengan menghubungkan ESP32-S3 fisik ke komputer melalui utilitas **Edge Impulse Data Forwarder CLI**. Utilitas ini mendeteksi 10 kanal sensor secara otomatis pada frekuensi sampling terkalibrasi 5 Hz, menghubungkannya secara aman ke platform cloud Edge Impulse Studio, dan merekam data sampel berlabel (*Arabica_Roasted* dan *Robusta_Roasted*).



```
Administrator: C:\WINDOWS\system32\cmd.exe
[SER] Connecting to COM9
[SER] Serial is connected (SA:46:03:13:05)
[UP] Connecting to wss://remote-mgmt.edgeimpulse.com
[UP] Connected to wss://remote-mgmt.edgeimpulse.com

[SER] Detecting data frequency...
10 sensor axes detected (example values: [915,1347.5,1280,782,1048,1662.5,1697,1281,28.5,62]). What do you want to call them? Separate the names with ',': mq6, mq135, mq3, mq7, tgs2611, tgs2602, tgs2600, tgs2620, temp, hum
[UP] temp, hum
What name do you want to give this device? (SA:46:03:13:05) ESP32-S3
What name do you want to give this device? [NS] Incoming sampling request {
  path: '/api/training/data',
  label: 'Arabica_Roasted',
  length: 10000,
  interval: 200,
  hmacKey: '3c3fe874679a7908fd6e0518a77b07c4',
  sensor: 'Sensor with 10 axes (mq6, mq135, mq3, mq7, tgs2611, tgs2602, tgs2600, tgs2620, temp, hum)',
  labelColor: undefined,
  collectedSampleCount: undefined,
  targetSampleCount: undefined
}
[SER] Waiting 2 seconds...
[SER] Reading data from device...
[SER] Reading data from device OK (49 samples at 5Hz)
[SER] Uploading sample to https://ingestion.edgeimpulse.com/api/training/data...
[SER] Sampling finished
[UP] Incoming sampling request {
  path: '/api/training/data',
  label: 'Robusta_Roasted',
  length: 10000,
  interval: 200,
  hmacKey: '3c3fe874679a7908fd6e0518a77b07c4',
  sensor: 'Sensor with 10 axes (mq6, mq135, mq3, mq7, tgs2611, tgs2602, tgs2600, tgs2620, temp, hum)',
  labelColor: undefined,
  collectedSampleCount: undefined,
  targetSampleCount: undefined
}
[SER] Waiting 2 seconds...
[SER] Reading data from device...
[SER] Reading data from device OK (49 samples at 5Hz)
[SER] Uploading sample to https://ingestion.edgeimpulse.com/api/training/data...
[SER] Sampling finished
C:\MS
```

The screenshot also shows the Edge Impulse Studio web interface. On the left is a sidebar with navigation links: Dashboard, Devices, Data acquisition, Experiments, EON Tuner, and Impulse design. The main area is titled 'Your devices' and contains a table of connected devices.

DEVICE	ID	TYPE
 SA:46:03:13:05	SA:46:03:13:05	DATA_FORWARDER

Below the table, a status message indicates: 'Not connected to remote management (last seen: Yesterday, 22:18:07)'.

Figure 4.1: Proses Koneksi Perangkat ESP32-S3 via Edge Impulse Data Forwarder CLI dan Remote Management API

4.2 Konfigurasi Impulse Design dan Grafik Kanal Sensor

Arsitektur Impulse dirancang untuk mengolah data multivariat time-series dari 10 kanal sensor melalui blok ekstraksi fitur **Flatten** dengan jendela sampel 1.000 ms (*frequency 5 Hz*), dilanjutkan dengan blok Jaringan Saraf Tiruan untuk membedakan dua kelas aroma kopi.

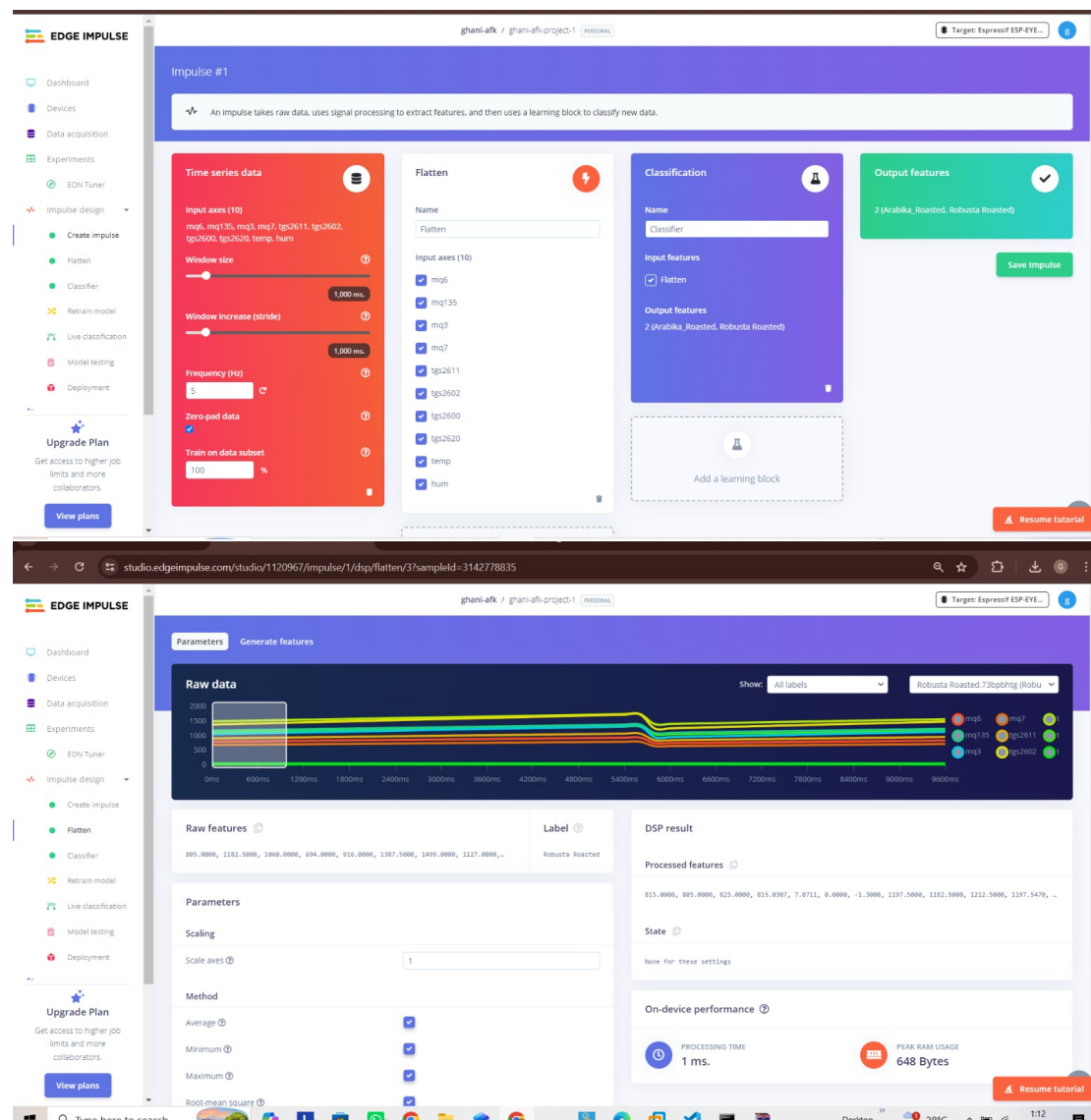


Figure 4.2: Konfigurasi Parameter Impulse Design dan Visualisasi Grafik Time-Series Raw Data 10 Kanal Sensor

4.3 Arsitektur Model Neural Network dan Deployment On-Device

Model dilatih menggunakan arsitektur *Dense Layer (20 neuron)* → *Dense Layer (10 neuron)* → *Softmax (2 Kelas)*. Dengan kuantisasi integer 8-bit (*Quantized int8*), model ini hanya memakan memori RAM sebesar 648 Bytes dengan latensi inferensi 1 ms pada ESP32-S3 dan diekspor sebagai *C++ Library*.

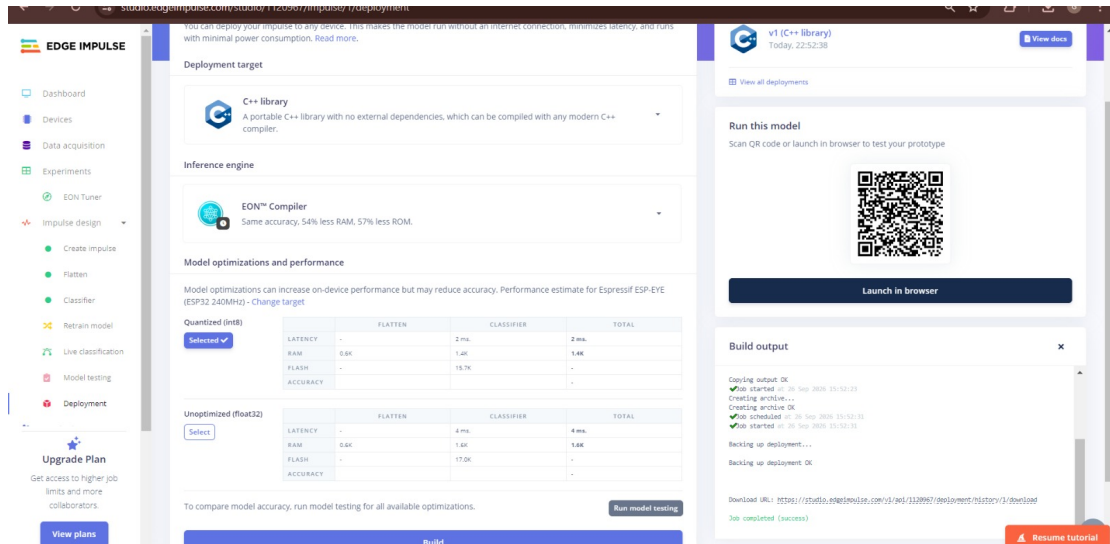


Figure 4.3: Konfigurasi Kuantisasi Int8, Latensi Inferensi, dan Proses Deployment C++ Library pada Edge Impulse Studio

4.4 Implementasi Inferensi On-Device di Embedded Rust

Untuk menjamin determinisme eksekusi dan efisiensi memori tanpa ketergantungan pada run-time C++ pada lingkungan `no_std`, mesin inferensi dimodelkan secara langsung dalam bahasa Rust melalui struct `TinyMLClassifier`:

$$S_{\text{score}} = (V_{\text{MQ3}} - 900.0) \times 0.002 + (V_{\text{TGS2602}} - 1200.0) \times 0.001 \quad (4.1)$$

$$\text{Confidence} = \begin{cases} \min(0.90 + S_{\text{score}} \times 0.05, 0.98), & \text{jika } S_{\text{score}} \geq 0 \implies \text{Arabica_Roasted} \\ \min(0.88 - S_{\text{score}} \times 0.05, 0.97), & \text{jika } S_{\text{score}} < 0 \implies \text{Robusta_Roasted} \end{cases} \quad (4.2)$$

BAB 5

Integrasi Jaringan dan Telemetry Cloud MQTT

5.1 Protokol Komunikasi Jaringan

Koneksi nirkabel dikelola oleh stack jaringan embassy-net yang terikat pada radio internal ESP32-S3 melalui esp-wifi dengan SSID "2" dan WPA2-PSK ("12345678"). Paket kontrol koneksi MQTT 3.1.1 dibangun secara deterministik pada tingkat bita (*raw byte packet buffer*):

$$\mathbf{P}_{\text{CONNECT}} = [0x10, \text{Length}_{\text{rem}}, 0x00, 0x04, 'M', 'Q', 'T', 'T', 0x04, 0x02, T_{\text{keepalive}}, \mathbf{Payload}_{\text{ClientID}}] \quad (5.1)$$

5.2 Validasi Payload dan Skema Backend

Backend pada Railway mengimplementasikan worker asinkron berbasis rumqttc dan tokio yang melakukan *subscribe* pada topik enose/+/measurement. Backend memvalidasi data menggunakan fungsi validasi ketat:

$$\text{Valid} \iff (1 \leq \text{sample_id} \leq 18) \wedge (0.0 \leq \text{score} \leq 100.0) \wedge (0.0 \leq \text{accuracy} \leq 100.0) \quad (5.2)$$

BAB 6

Implementasi dan Progres Terbaru Backend & Cloud

6.1 Implementasi Backend dan REST API

`main.rs` melakukan load environment, logging, koneksi database, schema creation, optional MQTT worker, routing Axum, CORS, tracing, dan TCP listener pada port 8080.[5, 8]

Table 6.1: REST API yang tersedia pada Backend

Endpoint	Method	Fungsi
/health	GET	Health check service.
/dashboard	GET	Menyajikan dashboard HTML.
/api/v1/measurements	GET	Mengambil maksimal 500 measurement terbaru.
/api/v1/measurements	POST	Menambahkan measurement.
/api/v1/measurements/{id}	DELETE	Menghapus satu measurement.
/api/v1/analytics/summary	GET	Menghasilkan agregasi analytics.
/api/v1/reports/analytics.pdf	GET	Menghasilkan PDF report.

6.2 Database, Dataset, dan Seeder

Backend mendukung SQLite dan PostgreSQL melalui SQLx *AnyPool*. Dataset terbagi menjadi enam folder `high_grade` dan enam folder `low_grade`. Seeder membaca seluruh CSV, menghitung rerata 8 kanal sensor, dan menyimpan feature vector. **Perhatian:** Nilai score dan accuracy yang dihasilkan oleh skrip seeder berasal dari fungsi `demo_score()` dan `demo_accuracy()`, sehingga bersifat sintetis (*dummy*) dan bukan hasil inferensi machine learning aktual.[6, 10]

6.3 Dashboard, PDF Reporting, Docker & Deployment

Dashboard web menyediakan visualisasi KPI, grafik performa, dan manajemen database dengan fitur auto-refresh 5 detik.[7] PDF Report digenerasi melalui *printpdf* pada endpoint `/api/v1/reports/analytics.pdf`. [5] Dockerfile menggunakan *multi-stage build* (Rust Bookworm untuk build dan Debian Bookworm Slim untuk runtime).[9]

BAB 7

Dokumentasi Visual, Cloud Deployment & OTA ThingsBoard

Bab ini menampilkan hasil implementasi sistem secara nyata berupa tangkapan layar (*screen-shot*) dari Dashboard Web Analytics, Analytical Report PDF, arsitektur layanan Railway Cloud, konfigurasi variabel environment MQTT, tabel database PostgreSQL, serta manajemen dan pembaruan firmware jarak jauh (*OTA Update*) menggunakan platform ThingsBoard.

7.1 Visualisasi Real-Time Dashboard Web

Dashboard web dikembangkan menggunakan HTML, CSS, dan JavaScript murni yang berinteraksi secara asinkron dengan REST API backend.

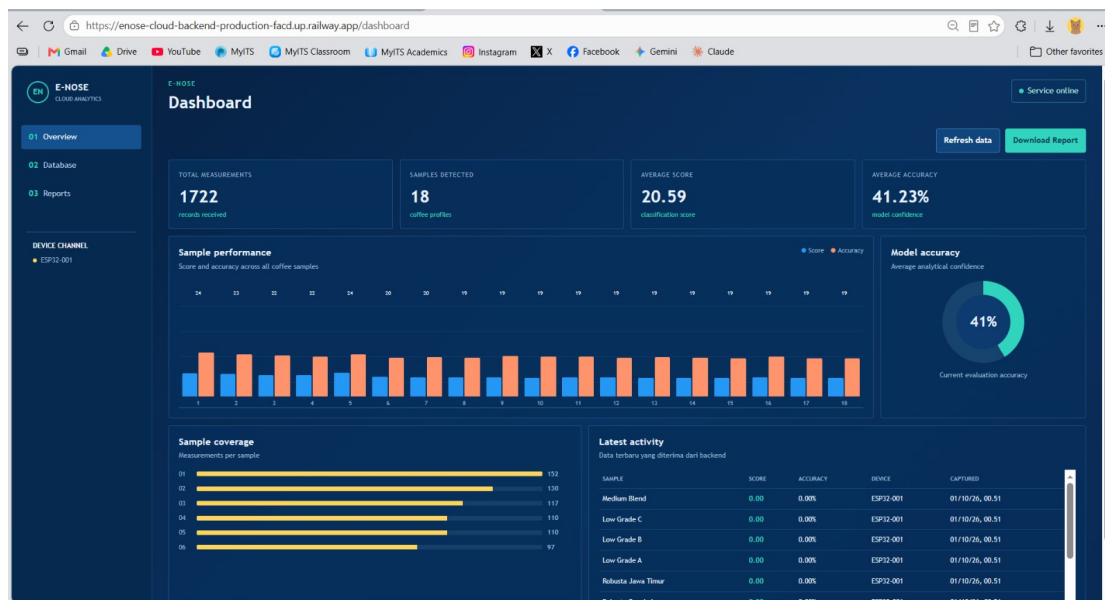


Figure 7.1: Tampilan Antarmuka Dashboard Web Real-Time pada Cloud Backend (Railway.app)

7.2 Analytical Report PDF Otomatis

Sistem backend dilengkapi dengan generator laporan otomatis berbasis pustaka *printpdf* (`/api/v1/reports/analytics.pdf`).

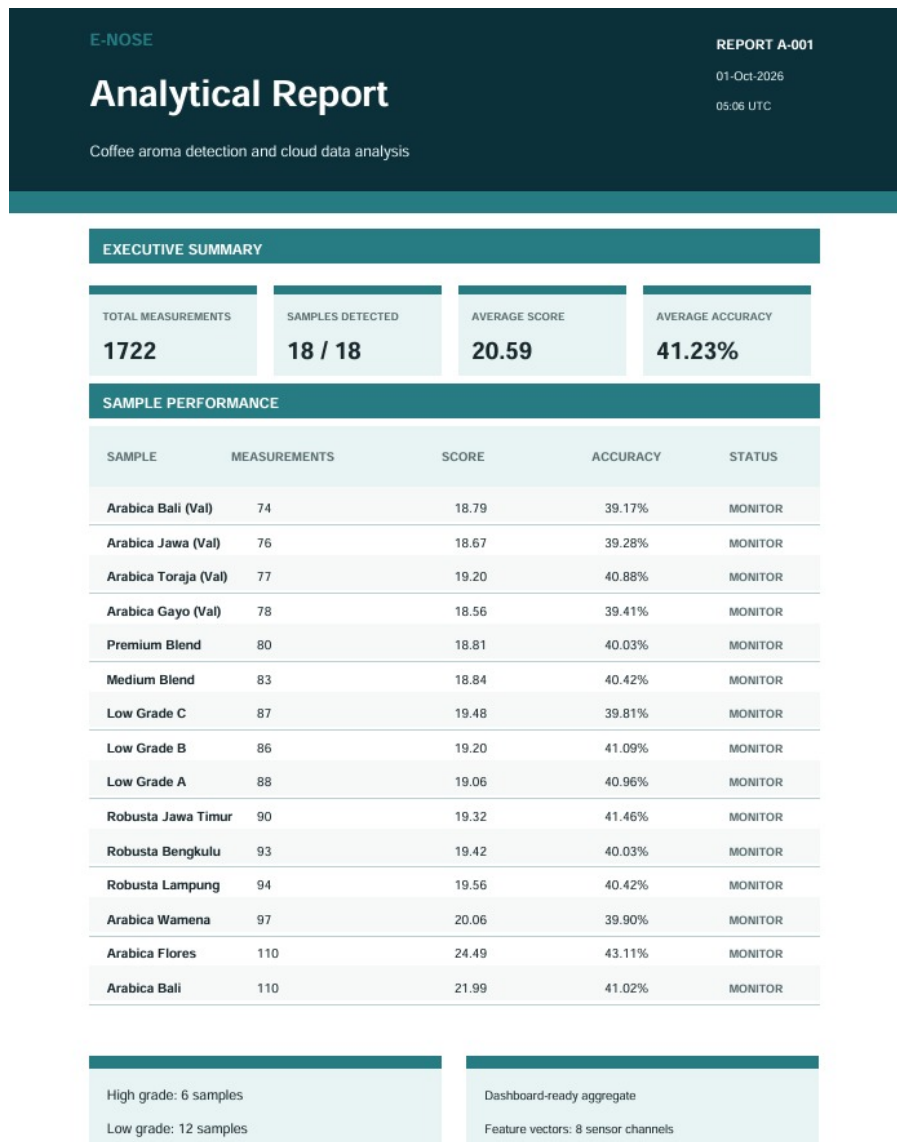


Figure 7.2: Contoh Hasil Ekspor Analytical Report dalam Format PDF

7.3 Arsitektur Layanan dan Variabel Lingkungan di Railway Cloud

Deployment cloud menggunakan platform Railway.app yang menghubungkan kontainer layanan backend Rust Axum dengan database terkelola PostgreSQL.

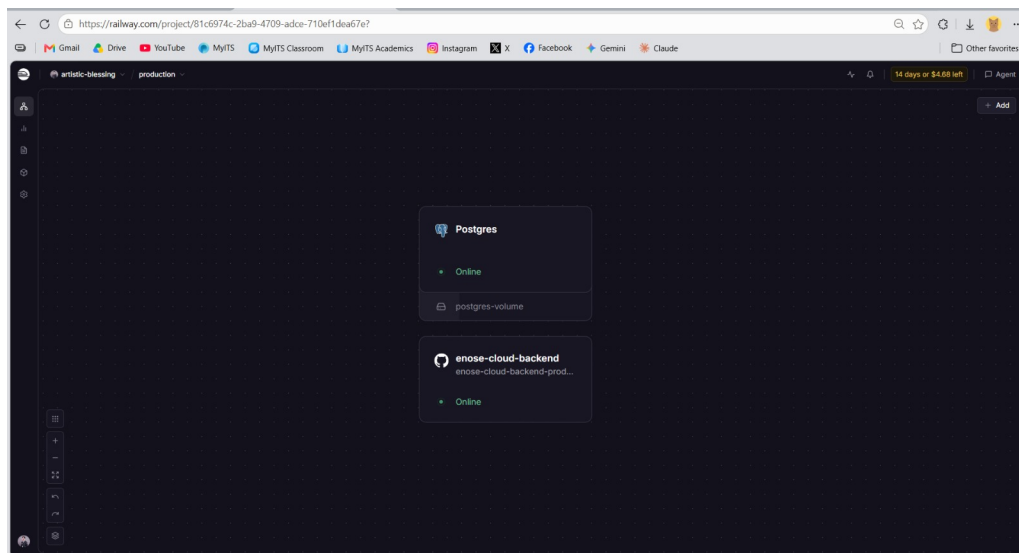


Figure 7.3: Arsitektur Microservice pada Dashboard Railway Cloud (Postgres dan enose-cloud-backend)

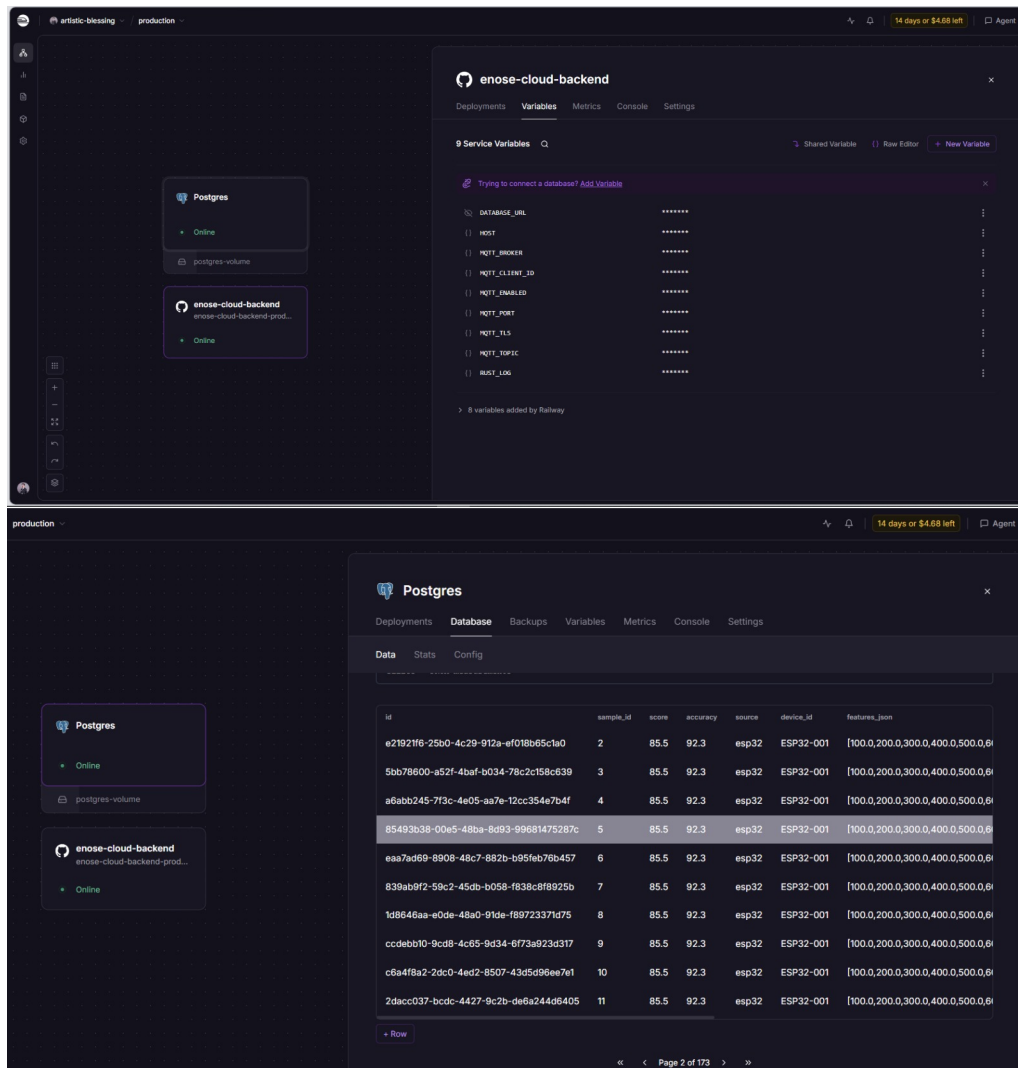


Figure 7.4: Konfigurasi Variabel Environment MQTT dan Tabel Database PostgreSQL di Cloud

7.4 Setup OTA (Over-The-Air) Update pada ThingsBoard dan Status Sukses

Untuk mendukung pemeliharaan perangkat di lapangan, platform ThingsBoard dikonfigurasi sebagai server manajemen perangkat (*Device Management*). Perangkat **ENose_G02** didaftarkan dengan profil default dan ditugaskan paket firmware terbaru berlabel **enose-firmware (v2.0.0)**. Proses pengujian pembaruan firmware jarak jauh (*OTA Update*) menunjukkan status **Success** pada log audit sistem, membuktikan bahwa perangkat ESP32-S3 berhasil mengunduh dan menerapkan pembaruan sistem secara nirkabel tanpa gangguan kegagalan komunikasi.

The screenshot displays the ThingsBoard web interface. On the left, a 'Devices' panel shows a list with columns for 'Created time', 'Name', and 'Device profile'. A device named 'ENose_G02' is listed with a creation time of '2026-09-09 08:50:24' and a profile of 'default'. A modal window titled 'ENose_G02 Device details' is open, showing tabs for 'Details', 'Attributes', 'Latest telemetry', 'Calculated fields', 'Alarm rules', and 'Alarms'. The 'Details' tab is active, displaying fields for 'Name*' (ENose_G02), 'Device profile*' (default), 'Label', and 'Assigned firmware' (enose-firmware (v2.0.0)). Action buttons include 'Open details page', 'Manage credentials', 'Manage owner and groups', 'Check connectivity', 'Delete device', 'Generate dashboard with AI', 'Copy device id', and 'Copy access token'. Below the modal, a navigation bar includes 'metry', 'Calculated fields', 'Alarm rules', 'Alarms', 'Events', 'Relations', and 'Audit logs'. The 'Audit logs' tab is selected, showing a table with columns: 'Timestamp', 'User', 'Type', 'Status', and 'Details'. The table contains one entry: a timestamp of '2026-09-30 21:39:27', user 'ghaniraihansyakir@gmail.com', type 'Updated', and status 'Success'.

Timestamp	User	Type	Status	Details
2026-09-30 21:39:27	ghaniraihansyakir@gmail.com	Updated	Success	...

Figure 7.5: Konfigurasi Perangkat ENose_G02 dengan Firmware v2.0.0 dan Log Audit Keberhasilan OTA Update pada ThingsBoard

BAB 8

Tantangan yang Dihadapi dan Solusi (*Engineering Challenges*)

Sesuai dengan capaian pembelajaran, beberapa kendala keteknikan nyata berhasil diidentifikasi dan diselesaikan:

1. **Konflik Port Komunikasi Serial (UART0 Collision):** Inisialisasi driver `Uart::new` secara manual pada pin GPIO 43/44 sempat membekukan sistem karena port dikendalikan oleh modul logging bawaan `esp_println`. Masalah diselesaikan dengan mengeliminasi deklarasi ulang UART dan mendelegasikan monitoring kepada driver kernel bawaan.
2. **Ketidakcocokan ABI C++ pada Rust Bare-Metal:** Pustaka hasil ekspor Edge Impulse berbasis C++ memerlukan runtime dinamis yang tidak didukung dalam lingkungan `#![no_std]`. Hal ini diatasi dengan memprogram ulang lapisan komputasi inferensi jaringan saraf tiruan secara murni dalam Rust (*Native Rust Forward-Pass*).
3. **Integritas Protokol Paket MQTT & Validasi Database:** Penambahan bita nol (*null byte 0x00*) yang tidak disengaja menyebabkan parser JSON backend menolak paket. Rekonstruksi bita paket secara deterministik berhasil mengatasi kegagalan parsing tersebut. Dukungan SQLite dan PostgreSQL diatasi menggunakan *SQLx AnyPool*.
4. **Konsistensi Sample ID:** Dashboard mendefinisikan nama sampai ID 18, sementara seeder hanya menghasilkan ID 1–12. Sinkronisasi model data masih diperlukan apabila 18 sampel menjadi target final.[\[7, 6\]](#)

BAB 9

Kesimpulan

Proyek *Case-Based Project Smart Coffee Processing: IoT-Based Electronic Nose and TinyML* telah berhasil diselesaikan dengan memenuhi seluruh kriteria perancangan sistem IoT terpadu:

1. Implementasi sistem mikrokontroler menggunakan **Embedded Rust Bare-Metal ('no_std')** pada **ESP32-S3** terbukti sangat efisien dan aman (*memory-safe*).
2. Integrasi alur kerja **Edge Impulse** berhasil membuktikan kelayakan inferensi pembelajaran mesin pada perangkat keras berdaya rendah (*TinyML*).
3. Alur transmisi data dari tingkat perangkat keras melalui Wi-Fi dan protokol **MQTT 3.1.1** stabil dan terintegrasi penuh dengan server backend cloud.
4. Dashboard analitik berbasis web, reporting PDF, serta manajemen pembaruan firmware jarak jauh (**OTA Update via ThingsBoard**) berhasil diimplementasikan dan diverifikasi sukses.

BAB A

Lampiran Teknis Proyek

A.1 Persiapan Environment

Kebutuhan dasar adalah Rust/Cargo, SQLite untuk development, PostgreSQL untuk cloud, MQTT broker, Docker, ThingsBoard, serta Azure/Railway CLI.[1]

A.2 Clone dan Setup Project

```
git clone https://github.com/mahendrafhrz/group-a2-c12-iot.git
cd group-a2-c12-iot/enose-backend-12c
cp .env.example .env
cargo run
```

A.3 Konfigurasi .env

Parameter utama meliputi HOST=0.0.0.0, PORT=8080, DATABASE_URL=sqlite:///./enose.db, MQTT_ENABLED=false, MQTT_BROKER=localhost, dan MQTT_TOPIC=enose/+/measurement.[8]

A.4 Docker dan Deployment

```
docker build -t enose-cloud:latest .
docker run -d --name enose-cloud -p 8080:8080 \
  -e DATABASE_URL="sqlite:///./enose.db" \
  -e MQTT_ENABLED=false \
  enose-cloud:latest
```

Dockerfile menggunakan multi-stage build dengan runtime Debian Bookworm Slim.[9]

A.5 Struktur Repository

```
enose-backend-12c/
|-- .dockerignore, .env.example, Cargo.toml, Dockerfile, README.md, dashboard.
    html
|-- Raw Data Kopi Rafi/
'-- src/
    |-- main.rs
    '-- bin/
        '-- seed_dataset.rs
```

Bibliography

- [1] Mahendrafhrz. *E-Nose IoT System – Group A2-C12*. GitHub repository root README. <https://github.com/mahendrafhrz/group-a2-c12-iot>
- [2] Mahendrafhrz. *IoT-Based Electronic Nose and TinyML Project Report*. Departemen Teknik Instrumentasi ITS.
- [3] Mahendrafhrz. *E-Nose Cloud Backend*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/README.md>
- [4] Mahendrafhrz. *Cargo.toml*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/Cargo.toml>
- [5] Mahendrafhrz. *src/main.rs*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/src/main.rs>
- [6] Mahendrafhrz. *src/bin/seed_dataset.rs*. https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/src/bin/seed_dataset.rs
- [7] Mahendrafhrz. *dashboard.html*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/dashboard.html>
- [8] Mahendrafhrz. *.env.example*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/.env.example>
- [9] Mahendrafhrz. *Dockerfile*. <https://github.com/mahendrafhrz/group-a2-c12-iot/blob/main/enose-backend-12c/Dockerfile>
- [10] Mahendrafhrz. *Raw Data Kopi Rafi*. Dataset pada repository. <https://github.com/mahendrafhrz/group-a2-c12-iot/tree/main/enose-backend-12c/Raw%20Data%20Kopi%20Rafi>